

Entrevista Técnica — Agência MundoAfora (desenhe no draw.io)

Como funciona este exercício. Abaixo está uma **conversa** entre a equipe e o cliente do sistema **Agência MundoAfora** (dono de uma agência de viagens). **Leia com atenção** e, a partir do que as pessoas dizem, **desenhe um diagrama de classes** no draw.io (ou app.diagrams.net). Não é para programar — é para **modelar**.

 **Seu diagrama precisa mostrar os quatro pilares da OO e os três tipos de relacionamento** (associação, agregação e composição). No fim há a **lista do que entregar**, um **lembrete da notação** e um **gabarito** (para o professor).

A entrevista

Gerente: Pessoal, vamos organizar o sistema da **Agência MundoAfora**. Vou trazer o cliente pra ele explicar, do jeito dele, o que o sistema precisa ter. Anotem tudo e depois a gente transforma em diagrama.

Cliente: No sistema **Agência MundoAfora**, a peça central é **Cliente**. Tem dois tipos: o **ClienteComum** (o comum) e o **ClientePremium**, que **inclui traslado e seguro-viagem**. No fundo, os dois são um tipo de **Cliente** — todos têm nome e documento e compartilham a mesma base.

Dev sênior: Entendi. Então tem uma parte **igual pra todos** e uma parte **específica** de cada tipo. Guardem isso — cheira a uma base comum com dois tipos que herdam dela.

Cliente: Isso. E o comportamento muda por tipo: olhando o **valorComExtras()** de cada um, pro **ClienteComum** paga só o pacote; já pro **ClientePremium** inclui traslado e seguro no valor. A **mesma** operação, **resposta** **diferente conforme o tipo**.

Analista de qualidade: Um ponto sério: o número de vagas e o valor não podem ser mexidos por fora (vagas nunca abaixo de zero). Ninguém pode mexer nesses dados **por fora** — tem que ser por **operações controladas**. E tem uma regra que **NÃO pode falhar: não pode reservar mais vagas do que o pacote oferece**. Por isso o estado de **Reserva** só anda por operações (pendente → confirmada; ou cancelada), nunca "na mão".

Cliente: Agora as ligações. Um pacote tem MUITAS reservas; cada reserva é de UM cliente para UM pacote.

Cliente: Tem uma ligação de **parte-todo que nasce e morre junto: Pacote** ◆ **ItemPacote**. Cada item (voo, hotel, passeio) só existe dentro do pacote e some junto com ele. (pense em **composição** — losango cheio ◆)

Dev sênior: Não confunda com o outro caso: **Pacote** ◇ **Destino** é só um **agrupamento**. Os destinos existem por conta própria; o pacote só os agrupa no roteiro. Se **Pacote** sumir, **Destino** continua existindo — isso é **agregação** (losango vazio ◇), bem diferente da composição.

Gerente: Fechou. Com isso dá pra desenhar o modelo. Mãos à obra.

Dicas de leitura (pistas escondidas na conversa)

O que apareceu na conversa	Onde isso te leva
"os dois são um tipo de Cliente (mesma base)"	um tipo geral (base) e tipos específicos → herança
"a mesma operação valorComExtras() , resposta muda por tipo"	operação redefinida em cada tipo → polimorfismo
"O número de vagas e o valor não podem ser mexidos por fora (vagas nunca abaixo de zero)"	dado privado + operações → encapsulamento
"não pode reservar mais vagas do que o pacote oferece"	invariante : o estado muda só por operação que valida
" Pacote ◆ ItemPacote (nascem e somem juntos)"	composição (losango cheio ◆)
" Pacote ◇ Destino (Destino existe sozinho)"	agregação (losango vazio ◇)
"um pacote tem MUITAS reservas; cada reserva é de UM cliente para UM pacote"	associação com multiplicidade

O que você deve entregar (no draw.io)

Um **diagrama de classes** que contenha, de forma clara:

1. **Abstração** — uma classe **base** que não é criada diretamente (ex.: **Cliente**).
2. **Herança** — **ClienteComum** e **ClientePremium** ligados à base pelo triângulo vazio ➡.
3. **Encapsulamento** — atributos sensíveis como **privados** (−) e **operações públicas** (+) que os controlam (proteger o valor e o estado da **Reserva**).
4. **Polimorfismo** — a operação **valorComExtras()** declarada na base e **redefinida** em cada tipo.
5. **Os três relacionamentos**, cada um com sua **multiplicidade**:
 - **Associação** (linha/seta): um pacote tem MUITAS reservas; cada reserva é de UM cliente para UM pacote.
 - **Agregação** (losango **vazio** ◇): **Pacote** junta **Destinos** que existem sozinhos.
 - **Composição** (losango **cheio** ◆): **Pacote** é feita de **ItemPacotes** que nascem e morrem com ela.

Lembrete de notação (UML)

Elemento	Como desenhar
----------	---------------

Elemento	Como desenhar
Classe abstrata	nome em <i>itálico</i> + « abstract » (não se cria direto)
Operação abstrata	em <i>itálico</i> (cada subtipo redefine)
Visibilidade	– privado · + público · # protegido
Herança	linha com triângulo vazio ➞ apontando para a base
Associação	linha (com seta), com multiplicidade nas pontas
Agregação	losango vazio ◇ no lado do "todo"
Composição	losango cheio ◆ no lado do "todo"

Multiplicidades: **1** (exatamente um), **0..1** (zero ou um), ***** (muitos), **1..*** (um ou mais).

✓ Critério de "pronto"

- ☐ Há uma **classe base abstrata** e os dois tipos herdando dela.
- ☐ Nenhum atributo sensível está público; o valor e o estado da **Reserva** são protegidos.
- ☐ A operação **valorComExtras()** aparece na base e é **redefinida** em cada tipo.
- ☐ Aparecem os **três** relacionamentos: **associação**, **agregação** (◇) e **composição** (◆).
- ☐ Você consegue **explicar** por que **Pacote**→**ItemPacote** é composição e **Pacote**→**Destino** é agregação.
- ☐ Todas as ligações têm **multiplicidade**.

← Briefing do projeto · 📅 Plano de evolução